Imperative Programming IV (Prog4)
(Tree-like Data Structures and C Programming)
Lecture #1
Trees and recursion

Christophe Tollu

A209 - ct@lipn.univ-paris13.fr

Tuesday 20 January 2026

Course organization and schedule

Timetable

- Weeks 4 to 9, 12 and 13: Only 8 weeks!
- 1,5h Lecture (CM) + 1,5h Supervised practical work (TD) + 1,5h Lab (TP) per week.
- Timetable likely to change. Check it every day!

Assessment

Written exam (Pa)

- No midterm exam
- Three-hour final written exam (week ??).

Other forms of assessment (EvC)

- Homework assignment (C programming)
- Lab reports and/or tests in class (starting next week)

Combined assessment

- (EvC + 2*Pa)/3

Warning

- Conditions for success: assiduity and hard work!

Assessment (2nd chance)

Only written exam (Pa2)

- Two-hour written exam in June 2026

No combined assessment

- Pa (2nd chance)

Make sure you avoid 2nd chance

- Attend lectures (CM), practical work sessions (TD) and labs (TP)
- Ask questions if you do not understand, practice regularly
- Develop your skills in C programming

Prog4 on line

Moodle: Course's website

https://moodle.univ-spn.fr/course/view.php?id=15751

Online resource

We will post

- News and announcements
- Slides of lectures
- Practice sheets and (perhaps) solutions
- Assignments
- Pieces of advice
- ...

- Check website every other day

Prog4 on line (continued)

Moodle: Course's website

`https://moodle.univ-spn.fr/course/view.php?id=15751`

Registration

We will post

- You are registered with your group
- Report every omission, error or change

- First practice sheets will be available this afternoon.

Syllabus

Aims

- Get a good command of some tree-like data structures, their implementation and the basic algorithms to manipulate them
- Be able to analyze and implement classical algorithms on trees
- Develop your skills in algorithm design and (structured) C Programming

Highlights

- Binary trees
- Heaps
- Binary search trees
- Balanced trees
- ...
- Function pointers (C)

Prerequisites and connections to other courses

Required: Basic notions from Prog2 (N2) and Prog3 (N3)

- Recursion
- Pointers and memory allocation
- Arrays, Linked lists, Queues, Stacks
- Program analysis and certification
- Cost measures and Growth rates (Bachmann and Landau's notation)
- `make` command and `makefile` configuration files

Main connections: "Conception d'algorithmes" (N5) and "Algorithmes de graphes" (N5)

- Algorithm design and analysis
- Focus on algorithmic strategies: Greedy, Divide and conquer, Dynamic programming, Flooding algorithms

Basic bibliography

Algorithms

- Donald E. Knuth, The Art of Computer Programming (4 volumes), Addison-Wesley, 1968–
  [Volumes 1-3 are in their 2nd or 3rd editions, Volumes 1 and 3 of particular interest to us]
  https://en.wikipedia.org/wiki/The_Art_of_Computer_Programm
- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, Introduction to Algorithms, MIT Press, 1990
  [Now in its fourth edition, 3rd edition translated into French and published by Dunod]
  https://en.wikipedia.org/wiki/Introduction_to_Algorithms

Programming in C

- Claude Delannoy, Programmer en langage C, Eyrolles, 2016
- Brian Kernighan and Dennis Ritchie, The C Programming Language, Prentice Hall, 1978
  https://en.wikipedia.org/wiki/The_C_Programming_Language

Today

Overview of Binary trees

- Definition
- Implementation
- Basic (recursive) algorithms

Trees: a mathematical abstraction

Central role in the design and analysis of algorithms

- Trees are used to describe dynamic properties of algorithms, particularly of **recursive algorithms**
- (Many) explicit data structures are **concrete realizations** of trees

Computer application: File systems

- Files are kept in directories
- Directories are (recursively) defined as sequences of files and directories
- This definition is identical to the definition of a certain type of (rooted) tree

Recursive algorithms

- Recursive calls are organized in a (rooted) tree
- This tree-like structure of recursive calls is used to analyse the algorithm
- It is reflected in the recurrence equation for the cost of a divide-and-conquer algorithm, for instance

(Undirected) graphs and trees

Definitions : graph, path

- A set V of **vertices** (or **nodes**)
- A set E of **edges**. Each edge connects two distinct nodes
- A (simple) **path** is a sequence of pairwise distinct nodes in which successive nodes are connected.
- The **length** of a path is the number of edges in the path, *i.e.* the number of distincts nodes in the path minus 1

Definition : tree, size, forest

- A **tree** is a graph satisfying the following property : for any pair of nodes x and y, there is exactly one (simple) path connecting x to y
- The **size** of a tree is the number of nodes
- A **forest** is a disjoint set of trees

Rooted trees

Definition : rooted tree

- A **rooted tree** is either an empty tree or a non-empty tree with a distinguished node, called the **root**

Directed paths in a rooted tree

- As soon as a root is chosen, all edges **can be given a direction** (all edges pointing away from the root or all edges pointing towards the root, depending upon the application one has in mind)
- In a rooted tree, there is **exactly one (directed) path connecting the root to any other node**

Henceforth

- All trees will be rooted
- “Tree” will stand for “rooted tree”

(Rooted) trees (continued)

Convention

- Computer scientists usually draw rooted trees with **the root at the top**!

Terminology

- Every node but the root has exactly one node immediately “above” it, called its **parent**
- The nodes immediately “below” a given node, if any, are called its **children**
- The analogy with family trees is sometimes carried further: grandparent, grandchild, ascendant, descendant, sibling, etc.

Definition: Subtree

- Given a node x in a rooted tree, x and its descendants constitute a rooted tree called the **subtree rooted in** x

(Rooted) trees (continued)

Definitions: Leaf

- A **leaf** of a rooted tree is a node with no child

Definitions: Layer, height, width, size

- In a rooted tree, nodes appear in **layers** (or **levels**)
- Layer 0 has the root as its unique element, layer 1 is composed of the children of the root, layer 2 is composed of its grandchildren, and so on...
- The **height of a node** is the length of the path from the root to the node; it is the number of the layer in which it lies
- The **height of a tree** is the maximal height of its nodes.
- The **width of a tree** is the maximal cardinality of its layers
- The **size** of a rooted tree is the number of nodes

Ordered trees

Definition: Ordered tree

- An **ordered tree** is a rooted tree such that, at every node x, the children of x, if any, are ordered
- Ordered trees have a natural representation as **embedded "planar" trees**: when one draws such an ordered tree, one draws the children of a node from left to right

Definition: m-ary tree

- An m**-ary tree** is an ordered tree in which every node has (at most) m children
- Additional **external** childless nodes are often defined, which act as dummy nodes (fake children for nodes with fewer than m "real" children)
- This "trick" turns out to be useful when one considers data structures for m-ary trees (more on very soon)

Binary trees

Definition: Binary tree

- A **binary** tree is 2-ary tree with two distinct types of nodes
- **External nodes** have no children
- **Internal nodes** have exactly two children, which are either external or internal nodes
- The children of an internal node x are usually referred to as the left and right children of x
- The internal nodes are the genuine nodes of the binary tree, external nodes are dummy nodes
- The **size** of a binary tree is the number of internal nodes

Alternative (equivalent) definition

- A **binary tree** is either an external node or an internal node connected to an ordered pair of binary trees, called its left subtree and right subtree

Labelled binary trees: Implementation

Two-link implementation

- An **(internal) node** encapsulates a label and two links, respectively to its left and right subtrees
- A **NULL link** corresponds to a link to an external node
- A binary tree is **identified with its root**

```
/* Excerpt from binary_tree.h */
#include "item.h"

struct binary_tree {
  item label; /* label of the root */
  struct binary_tree * left; /* NULL if no left subtree */
  struct binary_tree * right; /* NULL if no right subtree */
};

typedef struct binary_tree *link;
```

Binary trees: Elementary properties

Two properties

1.- A binary tree with n internal nodes has $n+1$ external nodes

2.- A binary tree with n internal nodes has $2n$ links: $n-1$ links to internal nodes and $n+1$ links to external nodes

Proofs

1.- Induction on n.

2.- Easy consequence of 1.-

Binary trees: Path lengths

Definitions: Internal and external path length

- The **internal path length** of a binary tree is the sum of the levels of all its internal nodes
- The **external path length** of a binary tree is the sum of the levels of all its external nodes

Binary trees: Properties of path length

Three properties

1.- The external path length of a binary tree with n internal nodes is $2n$ greater than the internal path length

2.- The height of a binary tree with n internal nodes is at least $\lfloor \log_2 n \rfloor$

3.- The external path length of a binary tree with n internal nodes is at least $n \log_2(n/4)$ and at most $n(n-1)/2$

Proofs

1.- Induction on n.

2.- Worst case: degenerate tree with only one leaf. Best case: balanced tree with 2^i nodes at all levels i but the bottom one.

3.- Same worst and best cases as in 2.-

Tree traversals

Recursive depth-first traversals

- **Preorder**: Visit the node, then visit the left subtree, then visit the right subtree
- **Inorder**: Visit the left subtree, then visit the node, then visit the right subtree
- **Postorder**: Visit the left subtree, then visit the right subtree, then visit the node

(Recursive and iterative) algorithms for traversals

- Next week's class and lab

Tree traversal (inorder)

```c
/* Excerpt from binary_tree.c */

/** Traverses recursively a tree according to inorder */
void traverse_inorder_binary_tree(link h, void (*visit)(link)) {
  if (h == NULL) return;
  traverse_inorder_binary_tree(h->left, visit);
  (*visit)(h);
  traverse_inorder_binary_tree(h->right, visit);
}
```

Tree traversals (continued)

Level-order (bredth-first) traversal

- Visit the nodes layer by layer, respecting the order on the children of every node

Algorithms for level-order traversal

- Next week's class and lab

Coming soon

- Function pointers in C (next week)
- Priority queues and heapsort
- Binary search trees